

Ejercicio 01 K-Medias

Notebook:  01 K-medias.ipynb

Centros usados

Originales (Géron):

```
blob_centers = np.array(  
    [[ 0.2, 2.3],  
     [-1.5, 2.3],  
     [-2.8, 1.8],  
     [-2.8, 2.8],  
     [-2.8, 1.3]])  
  
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
```

Modificados:

```
blob_centers = np.array(  
    [[ 0.2, 2.3],  
     [-1.5, 2.3],  
     [-3.6, 2.8],  
     [-2.0, 1.0],  
     [-3.8, 0.5]])  
  
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])
```

Métrica	Original (Géron)	Blobs separados	Cambio
Inercia k=3	653.22	1476.05	sube (forzar 3 clusters cuesta más)
Inercia k=5	224.07	212.52	baja un poco
Inercia k=8	127.13	119.87	baja un poco
Silueta k=5	0.6555	0.7770	sube claro
Pico visible en curva de silueta	k=4 (~0.69)	k=4 en la gráfica, pero k=5 real (0.777) la supera	ver nota
Codo (anotación del código)	k=4	k=4 (misma anotación, hardcoded)	no concluyente

Análisis

En los datos de Géron, ¿por qué el codo “prefiere” k=4 si make_blobs usó 5 centros?

Porque 3 de los 5 centros verdaderos (los de $\text{std}=0.1$ en $x=-2.8$) están casi pegados entre sí, su separación real está muy cerca del umbral de fusión ($2 \times (\sigma_i + \sigma_j)$). K-means los trata efectivamente como una sola región densa en vez de tres, así que tanto el codo como la silueta “ven” 4 grupos naturales, no 5.

Con tus blobs separados, ¿el codo y la silueta coinciden en el mismo k? ¿Ese k es 5?

No coinciden limpiamente. La silueta sí se mueve a k=5 de forma clara y medible (0.777 vs. 0.699 en k=4) — es la señal más fuerte del cambio. El codo es más ambiguo: la caída de k=4 a k=5 (de ~600 a 212, según la curva) sigue siendo grande, así que a simple vista el “quiebre” de la curva todavía podría leerse en k=4.

Si el codo sigue en 4, ¿qué te falta mover (distancia entre centros vs. blob_std)?

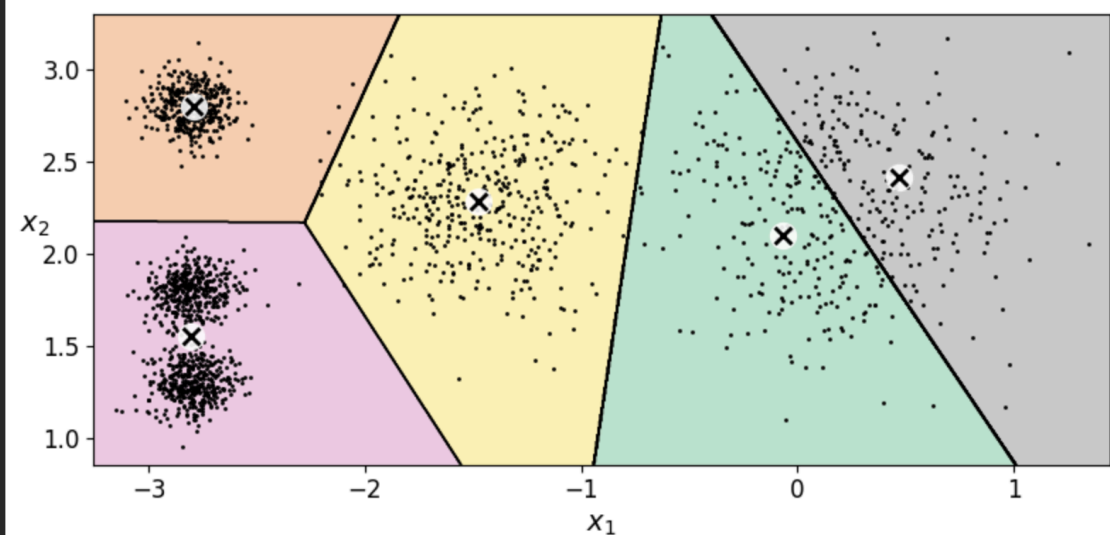
Lo que se tendría que ajustar es la distancia entre centros, no el blob_std. Los dos blobs que quedan en el límite son los que Géron ya tenía "separados" — el de $\text{std}=0.3$ en $(-1.5, 2.3)$ y el de $\text{std}=0.4$ en $(0.2, 2.3)$ — a solo 1.7 de distancia entre sí, contra un umbral de fusión de $2 \times (0.3 + 0.4) = 1.4$: apenas por encima, con muy poco margen. Alejar un poco más esos dos centros entre sí para que la distancia deje de estar tan pegada al umbral.

Evidencias

scatter, Voronoi ($k = 5$), codo y silueta — original

```
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
save_fig("voronoi_plot")
plt.show()
```

... Saving figure voronoi_plot



```
[1] from sklearn.metrics import silhouette_score

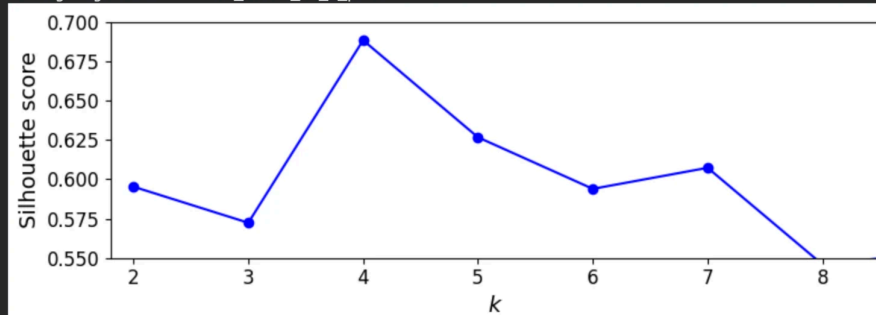
[1] silhouette_score(X, kmeans.labels_)

np.float64(0.65517642572828)

[1] silhouette_scores = [silhouette_score(X, model.labels_)
                      for model in kmeans_per_k[1:]]
```

```
[1] plt.figure(figsize=(8, 3))
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Silhouette score", fontsize=14)
plt.axis([1.8, 8.5, 0.55, 0.7])
save_fig("silhouette_score_vs_k_plot")
plt.show()
```

... Saving figure silhouette_score_vs_k_plot



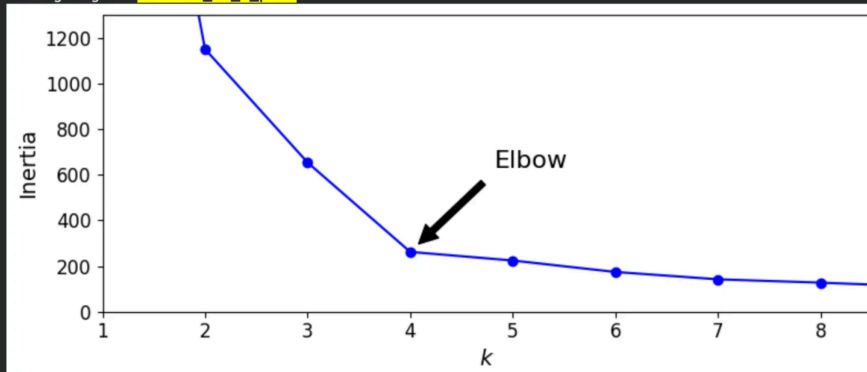
As you can see, this visualization is much richer than the previous one: in particular, although it confirms that $k = 4$ is a very good choice, but it also underlines the fact that $k = 5$ is quite good as well.

```

plt.figure(figsize=(8, 3.5))
plt.plot(range(1, 10), inertias, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Inertia", fontsize=14)
plt.annotate('Elbow',
             xy=(4, inertias[3]),
             xytext=(0.55, 0.55),
             textcoords='figure fraction',
             fontsize=16,
             arrowprops=dict(facecolor='black', shrink=0.1)
            )
plt.axis([1, 8.5, 0, 1300])
save_fig("inertia_vs_k_plot")
plt.show()

```

... Saving figure `inertia_vs_k_plot`



As you can see, there is an elbow at $k = 4$, which means that less clusters than that would be bad, and more clusters would not help much and might cut clusters in half. So $k = 4$ is a pretty good choice. Of course in this example it is not perfect since it means that the two blobs in the lower left will be considered as just a single cluster, but it's a pretty good clustering nonetheless.

scatter, Voronoi ($k = 5$), codo y silueta — Modificado

```

[59]
✓ 0 s
def plot_clusters(X, y=None):
    plt.scatter(X[:, 0], X[:, 1], c=y, s=1)
    plt.xlabel("$x_1$", fontsize=14)
    plt.ylabel("$x_2$", fontsize=14, rotation=0)

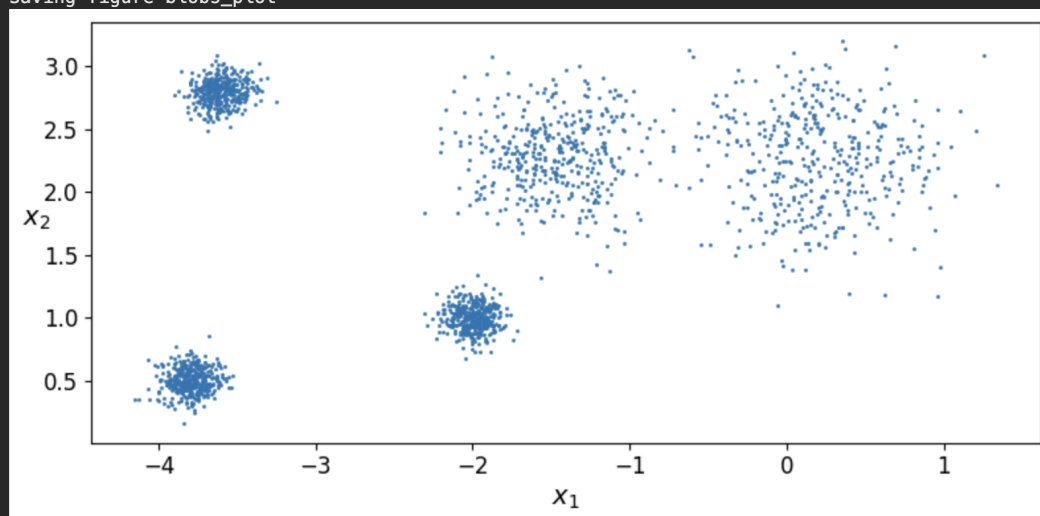
```

```

[60]
✓ 0 s
plt.figure(figsize=(8, 4))
plot_clusters(X)
save_fig("blobs_plot")
plt.show()

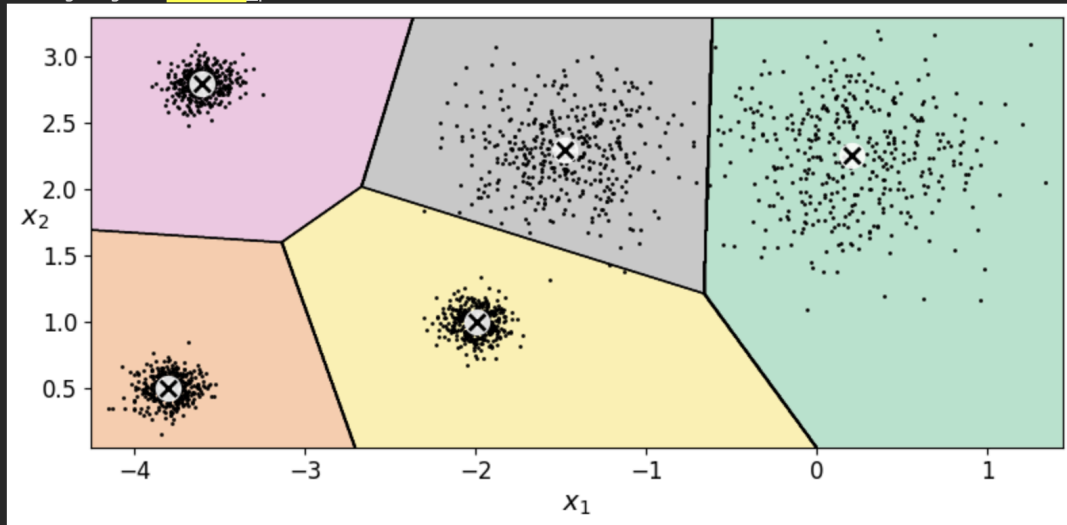
```

... Saving figure `blobs_plot`



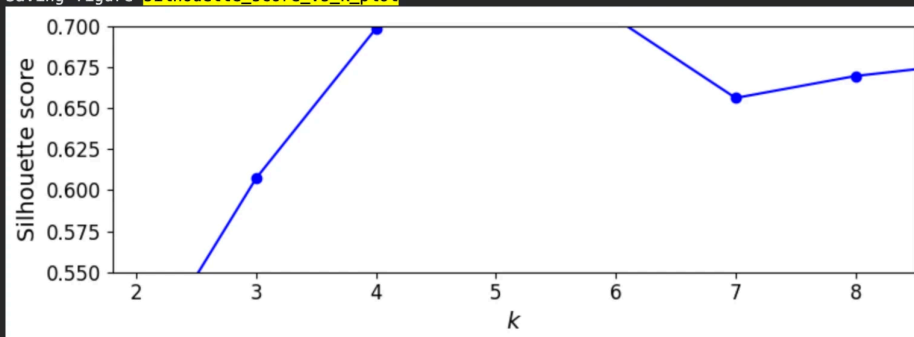
```
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
save_fig("voronoi_plot")
plt.show()
```

... Saving figure `voronoi_plot`



```
plt.figure(figsize=(8, 3))
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Silhouette score", fontsize=14)
plt.axis([1.8, 8.5, 0.55, 0.7])
save_fig("silhouette_score_vs_k_plot")
plt.show()
```

... Saving figure `silhouette_score_vs_k_plot`



As you can see, this visualization is much richer than the previous one: in particular, although it confirms that $k = 4$ is a very good choice, but it also underlines the fact that $k = 5$ is quite good as well.

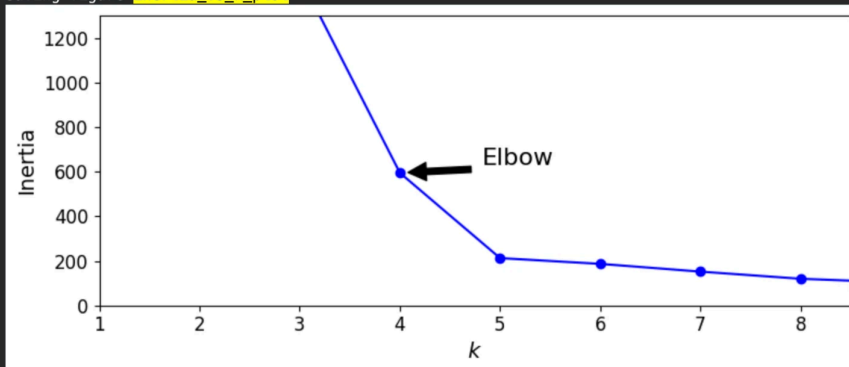
An even more informative visualization is given when you plot every instance's silhouette coefficient, sorted by the cluster they are assigned to and by the value of the coefficient. This is called a *silhouette diagram*.

```

plt.figure(figsize=(8, 3.5))
plt.plot(range(1, 10), inertias, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Inertia", fontsize=14)
plt.annotate('Elbow',
             xy=(4, inertias[3]),
             xytext=(0.55, 0.55),
             textcoords='figure fraction',
             fontsize=16,
             arrowprops=dict(facecolor='black', shrink=0.1)
            )
plt.axis([1, 8.5, 0, 1300])
save_fig("inertia_vs_k_plot")
plt.show()

```

... Saving figure `inertia_vs_k_plot`



As you can see, there is an elbow at $k = 4$, which means that less clusters than that would be bad, and more clusters would not help much and might cut clusters in half. So $k = 4$ is a pretty good choice. Of course in this example it is not perfect since it means that the two blobs in the lower left will be considered as just a single cluster, but it's a pretty good clustering nonetheless.